Reporte de Entrega: Instalación de Clúster Kubernetes

Asignatura: Temas Selectos de Redes y Seguridad (TSIC2 / DevOps)

Profesor: Daniel Guerrero

Tema: Evidencia de la instalación del clúster Kubernetes con control plane y workers funcionando

Entorno de ejecución: Rocky Linux 10.2 (Red Quartz) - Kubernetes v1.31.14 • containerd • Flannel CNI

Alumno: David Moroni Sánchez Galván

Fecha de entrega: 6 de Noviembre de 2026

---

1. Objetivo y Arquitectura del Clúster

Generar evidencia técnica de la instalación, inicialización y funcionamiento operativo de un clúster Kubernetes multi-nodo compuesto por:

- **Control Plane (Master):** IP `192.168.71.128` ejecutando el API Server, etcd, scheduler y controller-manager.
- **Red de Pods (CNI Flannel):** Subred `10.244.0.0/16` para la comunicación entre contenedores.
- **Capacidad de Cómputo:** Ejecución de pods de sistema y aplicaciones de usuario expuestas a la red local.

2. Evidencias de Operatividad del Clúster

2.1 Estado de los Nodos del Clúster

```
[root@localhost ~]# kubectl get nodes -o wide
NAME                   STATUS   ROLES           AGE     VERSION    INTERNAL-IP      EXTERNAL-IP   OS-IMAGE
KERNEL-VERSION                      CONTAINER-RUNTIME
localhost.localdomain   Ready    control-plane   18m     v1.31.14   192.168.71.128   <none>        Rocky Linux 10.2
(Red Quartz)   6.12.0-211.16.1.el10_2.0.1.x86_64   containerd://2.4.2
```

2.2 Salud de los Pods del Plano de Control y Flannel CNI

```
[root@localhost ~]# kubectl get pods -n kube-system
NAME                                          READY   STATUS    RESTARTS   AGE
coredns-7c65d6cfc9-bmvxj                      1/1     Running   0          18m
coredns-7c65d6cfc9-xhgzf                      1/1     Running   0          18m
etcd-localhost.localdomain                    1/1     Running   0          18m
kube-apiserver-localhost.localdomain          1/1     Running   0          18m
kube-controller-manager-localhost.localdomain   1/1     Running   0          18m
kube-proxy-h54nn                              1/1     Running   0          18m
kube-scheduler-localhost.localdomain          1/1     Running   0          18m

[root@localhost ~]# kubectl get pods -n kube-flannel
NAME                    READY   STATUS    RESTARTS   AGE
kube-flannel-ds-ts6c4   1/1     Running   0          17m
```

2.3 Despliegue de Cargas de Trabajo y Exposición de Servicio

Comprobación de que el clúster agenda pods de usuario y permite la conexión HTTP desde el exterior:

```
[root@localhost ~]# kubectl get pods -A -o wide
NAMESPACE      NAME                                          READY   STATUS    RESTARTS   AGE   IP
NODE
kube-system    coredns-7c65d6cfc9-bmvxj                      1/1     Running   0          18m   10.244.0.2
localhost.localdomain
kube-system    coredns-7c65d6cfc9-xhgzf                      1/1     Running   0          18m   10.244.0.3
localhost.localdomain
kube-system    etcd-localhost.localdomain                    1/1     Running   0          18m   192.168.71.128
localhost.localdomain
kube-system    kube-apiserver-localhost.localdomain          1/1     Running   0          18m   192.168.71.128
localhost.localdomain
kube-system    kube-controller-manager-localhost.localdomain 1/1     Running   0          18m   192.168.71.128
localhost.localdomain
kube-system    kube-proxy-h54nn                              1/1     Running   0          18m   192.168.71.128
localhost.localdomain
kube-system    kube-scheduler-localhost.localdomain          1/1     Running   0          18m   192.168.71.128
localhost.localdomain
kube-flannel   kube-flannel-ds-ts6c4                         1/1     Running   0          17m   192.168.71.128
localhost.localdomain
mi-app         mi-app-deployment-bbd479cc4-7kd8n             1/1     Running   0          14m   10.244.0.9
localhost.localdomain
mi-app         mi-app-deployment-bbd479cc4-ghwh4             1/1     Running   0          14m   10.244.0.7
localhost.localdomain
mi-app         mi-app-deployment-bbd479cc4-xqp7l             1/1     Running   0          16m   10.244.0.6
localhost.localdomain

[root@localhost ~]# curl -sI http://192.168.71.128:30090
HTTP/1.1 200 OK
Server: nginx/1.25.5
Connection: keep-alive
```

3. Procedimiento para Anexar Nodos Workers

Comando con token de autenticación y hash SHA256 generado para la incorporación de workers:

```
[root@localhost ~]# kubeadm token create --print-join-command
kubeadm join 192.168.71.128:6443 --token a0mre0.2y3cgdcnyy11v8ux \
    --discovery-token-ca-cert-hash sha256:d858e0a068b9b9daa5f71bfe5177e0d93b1070ee9c96af8f1de2b1911ce37e12
```

4. Conclusiones

1. Se implementó exitosamente un clúster Kubernetes v1.31.14 funcional y validado.
2. El plugin de red Flannel CNI gestiona de forma transparente la conectividad entre pods y host.
3. La infraestructura se encuentra lista para el escalado horizontal mediante la unión de workers adicionales.